Mathematical Abstractions of Hub Operations: Unified Framework (v1.2 - Research Edition)

Rafael Almeida (6068314)

2026-05-12

Contents

Mathematical Abstractions of the UPS Chelmsford Sort Training System **1**
Session Log . . . 2
How to Resume This Document . . . 2
Project State (as of Session 1) . . . 3
1. The Routing Function . . . 3
2. The ZIP Code Space as an Ultrametric . . . 4
3. The Belt Confusion Graph . . . 5
4. Weighted Question Selection as a Mixture Distribution . . . 6
5. The Missort Matrix as a Discrete Communication Channel . . . 7
6. Sorter Performance as a Statistical Process . . . 8
7. The Analytics Reducers as Monoid Homomorphisms . . . 10
8. The Overlay System as a Pointed Function Update . . . 11
9. The Knowledge Map as a Linear Functional . . . 11
10. [TRACKER] Snapshot Architecture and Temporal Data Integration . . . 12
11. [TRACKER] Sort Phases as a Temporal Markov Chain . . . 14
12. [TRACKER / DASHBOARD] PPH Quantization and Color Bands . . . 15
13. [MASTER TREND ANALYSIS] The Data Extraction Pipeline as a Functor . . . 16
14. [MASTER TREND ANALYSIS] Historical Trend Inference and Prediction Algorithms . . . 17
15. [MASTER TREND ANALYSIS] Door Turns as a Leading Indicator . . . 20
16. [MASTER TREND ANALYSIS] The Sort Summary as a Narrative + Heatmap . . . 20
17. [TRACKER + MASTER TREND] Three Timescales and Their Relationship . . . 21
18. [GLOBAL] Filtered Spaces and Stratification . . . 23
19. [LABEL TRAINING CERT] Knowledge Transfer as Information Geometry . . . 24
20. The Sort Chart as a Finite Automaton . . . 24
21. The Probabilistic Skill Model (Bayesian Extension) . . . 25
22. Comprehensive Summary Table of Mathematical Structures . . . 26
23. Integration Points and Feedback Loops (Future Architecture) . . . 28
24. Open Problems and Future Sessions . . . 29
25. How to Resume and Extend This Document . . . 30

Mathematical Abstractions of the UPS Chelmsford Sort Training System

Author: Rafael Almeida (6068314)
Project: LabelTrainingCertification + Tracker + Dashboard + MasterTrendAnalysis
Last updated: May 2026 (v1.2)

Status: Living document — each session extends the analysis. Read from the top before resuming.
Version Note: v1.2 incorporates deep-research findings on KDE bandwidth selection, Kalman filter cold-start strategies, DTW acceleration via Sakoe-Chiba bands, MLR feature scaling with mixed categorical/continuous variables, and CUSUM control limits for manufacturing quality control.

Session Log

Session	Date	Work Done
1	2026-05-12	Initial framework: routing function, ultrametric, confusion graph, weighted sampling, channel model, Bernoulli process, monoid reducers, overlay algebra, knowledge map
2	2026-05-12	Integration session: map Tracker (v2.9), Dashboard (v2.9.2), MasterTrendAnalysis (v2.0) into unified mathematical framework. Add snapshot architecture, temporal Markov chains, PPH quantization, curve fitting algorithms, data extraction functors, three timescale hierarchy.
3	2026-05-12	Review & correction: fix prefix metric boundary condition (d(z,z)=0), correct Fisher information formula, clarify Kalman filter state model, improve non-homomorphic aggregation explanation, strengthen monoid setup clarity, expand information-loss discussion. Version → v1.1
4	2026-05-12	Deep research & v1.2 refinement: KDE Sheather-Jones vs Silverman bandwidth tradeoffs; Kalman filter cold-start initialization strategies (Bayesian, Gaussian process optimization); DTW Sakoe-Chiba band constraint for speedup; MLR categorical+continuous feature scaling; CUSUM control limits for manufacturing (K, H parameters). Updated 5 major sections with production deployment guidance.

How to Resume This Document

This file is both a whitepaper and a session context file. Before beginning a new session of investigation:

1. Read the Session Log to know what has been established.
2. Read any sections marked `[OPEN]` — these are incomplete threads.
3. The **Project State** section below summarises the code structures being analysed; update it if the codebase has changed.
4. Add a row to the Session Log, then extend or refine the sections below.

Project State (as of Session 1)

The system is a single-page browser app (`index.html` + five JS modules) that trains UPS package sorters on routing decisions.

Key data structures in code:

Symbol	Code name	Description
Z	`TRUTH_RAW` (parsed into `_tmap`)	Set of ~41 000 ZIP codes with routing
B	`BELT_NAMES` indices 0–15	16 belt destinations
f	`_tmap : Map<zip, entry>`	The routing function
$\mathcal{F}$	`BELT_FAMILIES`	Confusion neighbourhood graph
$\mathcal{E}$	`sort_events` store	Sequence of sorter answer events
$\mathcal{O}$	`overlays` store	Finite set of routing corrections
w	`_cfg.stateWeights`	State weight vector
M	computed in `missortMatrix()`	The missort count matrix

Belt index reference:

```
0  Top Black      4  Bottom Red     8  Top Brown      12 Middle Black
1  Middle Blue    5  Top Blue       9  Bottom Black   13 Bottom Brown
2  Middle Red     6  Top Green     10  Top Red        14 AF1 (Middle Green)
3  Middle Yellow  7  Bottom Blue   11  Bottom Green   15 AF2 (Bottom Green)
```

CCHIL belts: {3, 4, 8} (Middle Yellow, Bottom Red, Top Brown)
Air belts: {14, 15} (AF1, AF2)

---

1. The Routing Function

1.1 Basic Structure

Let $Z \subset \{0, \ldots, 99999\}$ be the set of ZIP codes with defined routes, $|Z| \approx 41{,}000$. Let $B = \{0, 1, \ldots, 15\}$ be the belt index set. The **routing function** is:

$$f : Z \to B$$

This is a total function on its domain (every loaded ZIP maps to exactly one belt), but a *partial* function when viewed over all possible 5-digit strings $\{00000, \ldots, 99999\}$.

The pair (Z, B, f) constitutes a **finite relational structure** — specifically a functional binary relation (each $z \in Z$ has exactly one image under f).

1.2 The CCHIL Multivalued Extension

For ZIPs in the 13 CCHIL states (MS, IA, WI, MN, SD, ND, NE, LA, OK, CO, WY, AZ, NM), the app accepts any of the three CCHIL belts as correct:

$$f_{\text{CCHIL}} : Z_{\text{CCHIL}} \to \mathcal{P}(B), \quad f_{\text{CCHIL}}(z) = \{3, 4, 8\}$$

More precisely, the routing becomes a **multifunction** (a set-valued function):

$$\tilde{f} : Z \to \mathcal{P}(B), \quad \tilde{f}(z) = \begin{cases} \{f(z)\} & z \notin Z_{\text{CCHIL}} \\ \{3, 4, 8\} & z \in Z_{\text{CCHIL}} \end{cases}$$

The sorter's answer $a \in B$ is marked correct iff $a \in \tilde{f}(z)$. This is equivalent to defining an **acceptance relation** $\mathcal{A} \subseteq Z \times B$ where $(z, b) \in \mathcal{A}$ iff $b \in \tilde{f}(z)$.

1.3 Properties

- f is **not injective** in general: many ZIPs in different states share a belt destination. For example, `f⁻¹(0)` (Top Black, the PD-01 belt) is a large preimage covering northeast ZIPs in MA, NH, ME.
- f is **not surjective** over B from every state: each state contributes ZIPs to only a subset of belts.
- The preimage partition $\{f^{-1}(b) : b \in B\}$ is a **partition** of Z into 16 (possibly empty) classes. This partition is the fundamental structure the sorter must learn.

[OPEN] Characterise the preimage sizes $|f^{-1}(b)|$ for each belt b. Belts with large preimages are easier to learn by frequency; belts with small preimages are easily missed and dominate the Missort Matrix.

---

2. The ZIP Code Space as an Ultrametric

2.1 Prefix Metric

ZIP codes, viewed as 5-character strings over $\{0, \ldots, 9\}$, admit a natural **prefix metric**. Define the *longest common prefix length*:

$$\ell(z_1, z_2) = \max\{k : z_1[1..k] = z_2[1..k]\}$$

with the convention that $\ell(z, z) = 5$ (all five digits match). The metric is:

$$d(z_1, z_2) = \begin{cases} 0 & \text{if } z_1 = z_2 \\ 10^{5-\ell(z_1,z_2)} & \text{if } z_1 \neq z_2 \end{cases}$$

Equivalently, $d(z_1, z_2) = 10^{\max(5-\ell(z_1,z_2),0)} - \mathbf{1}[z_1 = z_2]$, but the case-wise definition is clearer. This satisfies the **ultrametric inequality**:

$$d(z_1, z_3) \leq \max(d(z_1, z_2),\ d(z_2, z_3))$$

which is strictly stronger than the triangle inequality, and importantly, satisfies the metric axiom $d(z, z) = 0$. The set (Z_{all}, d) is therefore an **ultrametric space**.

Geometric picture: Ultrametric spaces are "tree-like" — every triple of points forms an isoceles triangle where the two longer sides are equal. In our case, the tree is the *trie* (prefix tree) of ZIP codes, and the distance between two ZIPs equals the depth at which they diverge in the trie. This matches the physical geography: nearby ZIP codes share longer prefixes and route to the same or related belts.

2.2 Local Constancy of the Routing Function

The empirical observation is that f is **locally constant** at the 3-digit prefix level: ZIPs sharing the same 3-digit prefix almost always route to the same belt. More precisely:

$$d(z_1, z_2) \leq 100 \implies f(z_1) = f(z_2) \quad \text{(almost always)}$$

This local constancy is the reason the truth table was buildable from sort charts at all — the sort chart specifies routing at the SLIC/prefix level, and the pipeline expands it to individual ZIPs.

Formally, f approximately factors through the *quotient map* $\pi : Z \to Z/\sim_3$ where $z_1 \sim_3 z_2 \iff z_1[1..3] = z_2[1..3]$:

$$f \approx \bar{f} \circ \pi$$

where $\bar{f} : Z/\sim_3 \to B$ is the *prefix routing function*. The ~800 3-digit prefixes in active use cover all 41 000 ZIP entries.

[OPEN] Quantify the "exceptions to local constancy" — ZIPs within the same 3-digit block that route to different belts. These are precisely the cases where the sort chart has sub-prefix routing distinctions, and they are candidates for sorter confusion.

---

3. The Belt Confusion Graph

3.1 Definition

Define the **confusion graph** $G = (B, E)$ as a directed graph on the 16 belt nodes, where:

$$(b_i, b_j) \in E \iff b_j \in \texttt{BELT\_FAMILIES}[b_i]$$

In code, `BELT_FAMILIES` is a lookup from belt index to a list of "confusable" neighbours. The graph G encodes domain knowledge about which belts are physically or visually similar.

Example: Top Black (0) is confusable with Middle Black (12) and Bottom Black (9) — they share a colour family. Middle Blue (1) is confusable with Top Blue (5) and Bottom Blue (7).

3.2 Role in the Quiz Engine

When generating a wrong-answer distractor set, the quiz selects answers from $\mathcal{F}(b^*) = \{b : (b^*, b) \in E\}$ (the out-neighbourhood of the correct belt b^*). This ensures distractors are *semantically hard*, not random.

From a learning theory perspective, this is related to the concept of **hard negative mining**: training on confusable pairs accelerates discriminative learning compared to random negatives.

3.3 Graph-Theoretic Properties

`BELT_FAMILIES` is not symmetric in general (if b_j confuses with b_i, it does not necessarily mean b_i confuses with b_j — the relation reflects specific operational confusion patterns). So G is a directed graph.

Define the **confusion classes** as weakly connected components of G. Belts in the same class form a "family" that sorters tend to confuse with each other. From the belt naming, the three natural families are: - **Black family:** Top Black, Middle Black, Bottom Black - **Blue family:** Top Blue, Middle Blue, Bottom Blue - **Air family:** AF1, AF2

[**OPEN**] Compute the chromatic number of the undirected version of G. A k-colouring assigns each belt a "colour class" such that confusable belts get different colours — this gives the minimum number of visually distinguishable categories needed to make belt identification unambiguous.

---

4. Weighted Question Selection as a Mixture Distribution

4.1 The Four-Pool Model

The quiz generates questions from four pools:

Pool	Symbol	Bucket size
Ground (regular)	P_g	$\sum_{s \notin \text{CCHIL}} w_s \cdot \lvert G_s \rvert$
CCHIL	P_c	$w_c \cdot \bar{n}$
Exception labels	P_e	$w_e \cdot \bar{n}$
Air sort	P_a	$w_a \cdot \bar{n}$

where G_s is the set of ZIPs in state s, w_s is the state weight, and:

$$\bar{n} = \frac{\sum_{s \notin \text{CCHIL}} w_s \cdot |G_s|}{|\{s : s \notin \text{CCHIL}\}|}$$

is the **average per-state contribution** used to normalise all special pools.

The question type is selected by sampling uniformly from the total pool $P_g + P_c + P_e + P_a$, giving probabilities:

$$\Pr[\text{type} = t] = \frac{P_t}{P_g + P_c + P_e + P_a}$$

This is a **categorical mixture distribution** over question types, and within each type a uniform draw from the corresponding candidate set.

4.2 The Pre-v0.14 Bug and Its Fix

Before v0.14, air and exception bucket sizes were computed as $|\text{candidates}| \times w$ (e.g., $7 \times 20 = 140$) while the ground pool was $ $33 000. The resulting air share was $140/33{,}140 \approx 0.4\%$ — negligible at any weight.

The fix: replace the raw count with $w \times \bar{n}$, making all pools **commensurate**. With $w_a = 3$ and $\bar{n} \approx 868$:

$$P_a = 3 \times 868 = 2604 \quad \text{vs} \quad P_g \approx 33{,}000$$

giving Pr[air] $\approx 7.4\%$, which matches the intended behaviour.

Mathematical insight: The fix is equivalent to rescaling each pool by a common unit — the average state contribution. This is analogous to converting heterogeneous measures to a common denominator before mixing. The mixture distribution becomes *well-calibrated* with respect to the weight sliders.

4.3 The CCHIL Isolation Fix (v0.12)

Before v0.12, CCHIL states were included in the ground pool weighted by $w_s \times |G_s|$. Because CCHIL states collectively hold many ZIPs, their *aggregate* contribution dominated even when individual state weights were at default.

The fix isolates CCHIL into its own pool with a single weight w_c. This is equivalent to imposing **independence of CCHIL frequency from regional weight adjustments** — a desirable stochastic independence property. Formally, the pre-fix mixture was not a product measure; the fix makes the CCHIL component orthogonal to regional adjustments.

---

5. The Missort Matrix as a Discrete Communication Channel

5.1 Channel Model

A sorter can be modelled as a **discrete memoryless channel** with input alphabet B (the correct belt) and output alphabet B (the sorter's answer). The **channel transition matrix** is:

$$\mathbf{M} \in \mathbb{R}^{|B| \times |B|}, \quad M_{ij} = \frac{\text{count}(f(z) = b_i,\ \text{answer} = b_j)}{\text{count}(f(z) = b_i)}$$

$\mathbf{M}$ is a **right stochastic matrix**: each row sums to 1 (the sorter always gives some answer). A **perfect sorter** has $\mathbf{M} = \mathbf{I}_{16}$ (the identity). A **random guesser** has $M_{ij} = 1/16$ for all i, j.

The diagonal M_{ii} is per-belt accuracy. The off-diagonal entries M_{ij} (for $i \neq j$) are the missort rates.

5.2 Information-Theoretic Measures

Per-belt Shannon entropy: For belt b_i, the sorter's output distribution is the i-th row of $\mathbf{M}$. Its entropy:

$$H_i = -\sum_j M_{ij} \log_2 M_{ij} \in [0, \log_2 16] \text{ bits}$$

A perfect sorter has $H_i = 0$ for all i. A confused sorter has $H_i > 0$; maximum confusion gives $H_i = 4$ bits.

Mutual information: The mutual information between correct belt X and chosen belt Y is:

$$I(X;Y) = H(Y) - H(Y|X) = H(X) - H(X|Y)$$

$I(X;Y)$ measures how much the sorter's answer reveals about the correct belt. For a perfect sorter, $I(X;Y) = H(X)$ (full information). For a random guesser, $I(X;Y) = 0$.

Channel capacity: $C = \max_{p(X)} I(X;Y)$ bits per question — the maximum information a sorter can convey in a single routing decision, maximised over the input distribution. Estimating C empirically requires the full channel matrix.

5.3 The Missort Matrix in the App

In code, `missortMatrix(events)` computes raw counts. The app renders absolute heat colours (not row-normalised) because the count matrix is more operationally actionable — supervisors want to know "how many times" not just "what fraction." But the information-theoretic analysis requires the row-normalised version.

[OPEN] Add an entropy column to the belt analysis tab showing H_i per belt. High-entropy belts are a priority for coaching regardless of absolute accuracy, because entropy captures *systematic* confusion (the sorter has a wrong internal model), whereas low accuracy on a low-entropy belt may just mean the sorter doesn't know it — a simpler training problem.

---

6. Sorter Performance as a Statistical Process

6.1 Bernoulli Trials Model

Each quiz question for sorter k is modelled as a **Bernoulli trial** with success probability $\theta_k \in [0,1]$ (the sorter's true skill on the question distribution). Observed answers $(X_1, X_2, \ldots, X_n)$ are i.i.d. Bernoulli(θ_k).

The **maximum likelihood estimate** of θ_k is the sample accuracy:

$$\hat{\theta}_k = \frac{1}{n}\sum_{t=1}^{n} X_t = \frac{\text{correct}}{n}$$

6.2 Confidence Intervals

The app currently displays $\hat{\theta}_k$ without uncertainty. For small n (common early in a sorter's history), the Wald interval $\hat{\theta} \pm z_{\alpha/2}\sqrt{\hat{\theta}(1-\hat{\theta})/n}$ behaves poorly because it can produce intervals outside $[0,1]$ and undercovers at the tails.

The **Wilson score interval** is preferred:

$$\left[\frac{\hat{\theta} + \frac{z^2}{2n} \mp z\sqrt{\frac{\hat{\theta}(1-\hat{\theta})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}\right]$$

with $z = z_{\alpha/2} = 1.96$ for 95% confidence. This interval has correct coverage for all n and all θ, including θ near 0 or 1.

Practical implication: A sorter with 5 correct out of 5 ($\hat{\theta} = 100\%$) is displayed as "Proficient" but the 95% Wilson interval is $[0.54, 1.00]$ — the data are entirely consistent with a true skill of 54%. The threshold $n \geq 10$ in `worstSorter()` and \$n 5inworstBelt()‘ are reasonable minimum sample size guards.

[**OPEN**] Add Wilson lower-bound sorting as an alternative to raw accuracy in the sorter table — rank sorters by the lower endpoint of their Wilson interval rather than $\hat{\theta}$. This corrects for the optimism bias that makes new sorters with 3/3 appear above experienced sorters with 120/150.

6.3 Trend Detection: Half-Split Test

The app splits a sorter's events into first half and second half and shows the accuracy change. This is a rudimentary test of **non-stationarity** — is θ improving over time?

The proper statistical test is a **two-sample proportions test** (or equivalently, a chi-squared test of homogeneity on the 2×2 table of [correct/wrong] × [first/second half]):

$$z = \frac{\hat{\theta}_{\text{late}} - \hat{\theta}_{\text{early}}}{\sqrt{\hat{\theta}(1-\hat{\theta})(1/n_1 + 1/n_2)}}$$

under $H_0 : \theta_{\text{early}} = \theta_{\text{late}}$, where $\hat{\theta}$ is the pooled accuracy.

[**OPEN**] Implement the formal two-proportions z-test and display a p-value alongside the trend arrow. A threshold like $p < 0.05$ gives statistical confidence that the apparent trend is real rather than noise.

6.4 Session-Level Trend: A CUSUM Perspective

The sparkline chart plots session accuracy over time. This can be interpreted as a **CUSUM (cumulative sum control chart)**: if we define $S_n = \sum_{t=1}^{n}(X_t - \theta_0)$ for some target accuracy θ_0, then $S_n > H$ signals that the process has shifted above θ_0.

CUSUM is standard in industrial quality control — exactly the setting (package sorting as a manufacturing process) in which this system operates. A formal CUSUM implementation would detect skill improvements or degradations earlier than visual inspection of the sparkline.

CUSUM Control Limits (v1.2 research): CUSUM charts require two parameters:

1. **Reference value K:** The magnitude of shift to detect, typically set to half the distance from target to the shifted mean. Example: if target accuracy is $\theta_0 = 0.85$ and we want to detect a shift to 0.80 (5-point change), set $K = 0.025$.
2. **Decision interval H:** The control limit for cumulative deviation. Standard practice sets $H = 5\sigma$ (or $H = 4\sigma$ in sensitive applications), where $\sigma = \sqrt{\theta_0(1-\theta_0)/n}$ is the standard error per batch of questions. For a sorter with $n = 20$ questions per session and $\theta_0 = 0.85$:

$$\sigma \approx \sqrt{0.85 \times 0.15/20} \approx 0.045, \quad H = 5 \times 0.045 \approx 0.225$$

Recommendation: Implement CUSUM with $K = 0.025$ (detect 5-point shifts) and $H = 5\sigma$ (standard warehouse/manufacturing setting). Track cumulative sum per sorter over sessions, flag when $|S_n| > H$ as a significant trend.

---

7. The Analytics Reducers as Monoid Homomorphisms

7.1 Algebraic Setup

Let $\mathcal{E}^*$ be the set of all finite sequences of sort events, with concatenation $(\cdot)$ as the binary operation and the empty sequence ε as the identity. Then $(\mathcal{E}^*, \cdot, \varepsilon)$ is the **free monoid** over the event type $\mathcal{E}$.

Each analytics reducer function maps a sequence of events to a summary statistic. For example, `overviewStats : \mathcal{E}^* \to \text{Stats}` maps a sequence of events to a record $\text{Stats} = \{\text{total}, \text{correct}, \text{sorters}, \text{errRate}\}$. Define the merge operation on stats as:

$$\text{Stats}_1 \oplus \text{Stats}_2 = \{\text{total} : n_1 + n_2,\ \text{correct} : c_1 + c_2,\ \text{sorters} : \text{union}(s_1, s_2),\ ...\}$$

More generally, each summand is combined via an **associative, commutative operation** (addition for counts, union for sets). The identity element is:

$$\text{zero} = \{\text{total} : 0,\ \text{correct} : 0,\ \text{sorters} : \emptyset,\ ...\}$$

Then $(\text{Stats}, \oplus, \text{zero})$ is a **monoid**. The key property is:

$$\texttt{overviewStats}(e_1 \cdot e_2) = \texttt{overviewStats}(e_1) \oplus \texttt{overviewStats}(e_2)$$

This makes `overviewStats` a **monoid homomorphism** from $(\mathcal{E}^*, \cdot, \varepsilon)$ to $(\text{Stats}, \oplus, \text{zero})$. The homomorphism property says: aggregating the combined sequence is equivalent to aggregating each part separately and merging.

7.2 Why This Matters

The monoid structure implies the computation can be **parallelised via MapReduce**: partition the event log into chunks, reduce each chunk independently, then merge the partial results. This is the mathematical reason the analytics module is architected as pure reducers — the "no side effects in computation functions" design note in the source header is not just code hygiene, it is a structural requirement for the monoid homomorphism property to hold.

Practical consequence: If the event log grows to millions of records, incremental computation becomes possible: store partial aggregates and update only the new events, merging with the existing summary. The current IndexedDB schema could support this with a `last_aggregate_ts` marker.

7.3 Functions That Are Not Monoid Homomorphisms

Some analytics functions are *not* homomorphisms over the full monoid: - `worstBelt`, `worstSorter` — these involve sorting and `filter (total >= 5)`, which is a non-linear operation on the merged stats. - `missortMatrix` — the matrix itself is a homomorphic aggregate, but rendering top confusion pairs involves a sort, making the render step non-homomorphic.

These are **derived** from homomorphic aggregates, then post-processed. The pattern is: (1) compute the aggregate homomorphically, (2) apply a non-linear query to the aggregate. This is analogous to the SQL model: aggregation in `GROUP BY` is homomorphic; `ORDER BY` and `HAVING` are not.

---

8. The Overlay System as a Pointed Function Update

8.1 Algebraic Formulation

Let $\mathrm{Fun}(Z, B)$ be the set of all functions from Z to B. The original routing $f \in \mathrm{Fun}(Z, B)$. An **overlay** is a finite partial function $o : D_o \to B$ where $D_o \subset Z$ is a finite set of ZIPs being overridden.

Define the **overlay application operator** $\lhd$:

$$f \lhd o = \lambda z. \begin{cases} o(z) & \text{if } z \in D_o \\ f(z) & \text{otherwise} \end{cases}$$

This is the **pointwise override** — o takes precedence at its domain. The updated routing is $f' = f \lhd o$.

8.2 Categorical Interpretation

In the category of sets and partial functions, the operation $f \lhd o$ is a **pushout**. Specifically, given the inclusion $\iota : D_o \hookrightarrow Z$ and the functions f (restricted to D_o) and o, the pushout produces the function that agrees with o on D_o and with f on $Z \setminus D_o$.

8.3 Composability and Audit Trail

If multiple overlays $o_1, o_2, \ldots, o_k$ are applied sequentially, the result is:

$$f' = f \lhd o_1 \lhd o_2 \lhd \cdots \lhd o_k$$

The operation $\lhd$ is **not commutative** in general (later overlays override earlier ones on their shared domain). The app stores overlays as an append-only log (`overlays.jsonl`), which makes the history auditable — the exact sequence of overrides is preserved.

The set of all finite partial functions $D \to B$ with the $\lhd$ operation forms a **left-regular band** (an idempotent semigroup where $a \lhd a = a$ and $a \lhd b \lhd a = a \lhd b$). Associativity holds: $(f \lhd o_1) \lhd o_2 = f \lhd (o_1 \lhd o_2)$.

---

9. The Knowledge Map as a Linear Functional

9.1 Event Space Structure

Let the event space be $\mathcal{E} = \{(k, z, b_{\text{exp}}, b_{\text{act}}, c) : k \in K, z \in Z, b \in B, c \in \{0, 1\}\}$ where K is the set of sorter IDs.

A **filter** is a function $\varphi : \mathcal{E} \to \{0, 1\}$ that selects a subset of events. In the Knowledge Map, filters are state-based:

$$\varphi_S(e) = \mathbf{1}[\text{state}(e.z) \in S]$$

for a selected state set $S \subseteq \text{States}$, or:

$$\varphi_{\text{air}}(e) = \mathbf{1}[e.b_{\text{exp}} \in \{14, 15\}]$$

The **sorter accuracy under filter** φ for sorter k is:

$$a_k(\varphi) = \frac{\sum_{e:e.k=k} \varphi(e) \cdot e.c}{\sum_{e:e.k=k} \varphi(e)}$$

This is a **ratio of two linear functionals** (numerator and denominator are both linear in φ). As a function of the filter, a_k is a **rational linear functional** on the indicator function space.

9.2 The Ranking is a Partial Order

Given filter φ, the Knowledge Map produces a ranking $\sigma_\varphi : K \to \{1, \ldots, |K|\}$ by sorting sorters by $a_k(\varphi)$ descending. This ranking: - Depends continuously on φ (small changes to the filter cause small changes in $a_k(\varphi)$, modulo the discretisation at the 3-attempt minimum threshold). - Changes **discontinuously** when the relative order of two sorters swaps — these are the "phase boundaries" in the filter parameter space.

9.3 Filter Combinations as Set Operations

The app allows combining state selections and special categories. The combined filter $\varphi_{S,\text{CCHIL,air}}$ is:

$$\varphi(e) = \varphi_S(e) \ \vee \ \varphi_{\text{CCHIL}}(e) \ \vee \ \varphi_{\text{air}}(e)$$

This is the **logical OR** (union of event subsets). The filter algebra is the **Boolean algebra** $\mathcal{P}(\mathcal{E})$ under union, intersection, and complement — the Knowledge Map explores a slice of this algebra indexed by the state/category selectors.

[**OPEN**] Implement intersection filters: "sorters who are strong in CCHIL *and* Air" — this selects events matching both criteria in an AND sense, or equivalently, evaluates sorter accuracy separately per category and ranks by the minimum (weakest category) or product. This identifies sorters with balanced, broad knowledge versus specialists.

---

10. [TRACKER] Snapshot Architecture and Temporal Data Integration

10.1 The Three-Layer Data Model

The **Hub Operations Tracker** (v2.9, embedded in container v4.9) processes live sort data via three data sources:

1. **iGate Hub Summary** — Belt-level aggregates at a point in time: (Belt, LooseOutbound, BagsLinked, Gross Volume, etc.)
2. **iGate Employee Summary** — Per-employee per-belt scan counts with First/Last Scan Timestamps and cumulative hours.
3. **SOR (US-CHEMA)** — Three-sheet export: Summary (planned vs actual), Work Area Types (% vol distribution), Staffing (employee → area → hours).

A **snapshot** S_t at time t is a tuple:

$$S_t = (t, \mathcal{H}_t, \mathcal{E}_t, \mathcal{A}_t)$$

where: - t — elapsed time (minutes from sort start 18:00), inferred from max Last Scan Timestamp or SOR timestamp - $\mathcal{H}_t \in$ Hub — the hub-level record: $\{(b, \text{loose}_b, \text{bags}_b) : b \in B\}$ - $\mathcal{E}_t \in \text{Employee}^{|E|}$ — per-employee vector: $\{(k, b, n_{kb}, t_k^{\text{first}}, t_k^{\text{last}}, h_k) : k \in E, b \in B\}$ - $\mathcal{A}_t \in \text{Area}^{|Z|}$ — per-zone staffing: $\{(z, h_z^{\text{plan}}, h_z^{\text{actual}}) : z \in Z\}$

The **snapshot sequence** $\mathcal{S} = (S_{t_1}, S_{t_2}, \ldots, S_{t_m})$ with $0 = t_1 < t_2 < \cdots < t_m \leq 240$ (4-hour sort) forms a **discrete temporal observation**.

10.2 The iGate Scan Metrics Table

The **Per-Employee Productivity** table in the Tracker displays $\mathcal{E}_t$ with derived statistics per employee-belt pair:

$$\text{PPH}_{kb}^{(t)} = \frac{n_{kb}}{h_k}, \quad \text{Efficiency}_{kb}^{(t)} = \frac{\text{PPH}_{kb}^{(t)}}{\text{Target}_{kb}}$$

where Target_{kb} varies by belt and employee role. The **conditional color formatting** applies belt-family-specific thresholds:

- **Black family** (belts 0, 9, 12): threshold range $[\text{low}, \text{high}]$ in PPH
- **Blue family** (belts 1, 5, 7): distinct threshold range
- **Green family** (belts 6, 11, 14, 15): same as blue
- **Special belts** (3, 4, 8 CCHIL): unified threshold

This is **quantization** of a continuous variable (PPH) into a finite set of color classes $\{\text{red}, \text{amber}, \text{green}\}$, indexed by belt family. The quantization function is:

$$q_F : \mathbb{R}^+ \to \{\text{red}, \text{amber}, \text{green}\}, \quad q_F(\text{PPH}) = \begin{cases} \text{red} & \text{if PPH} < \text{low}_F \\ \text{amber} & \text{if low}_F \leq \text{PPH} < \text{high}_F \\ \text{green} & \text{if PPH} \geq \text{high}_F \end{cases}$$

The **information loss** from quantization can be estimated heuristically: if PPH values span a range (e.g., 10–50 packages/hour, a span of 40), quantizing into 3 bands loses approximately $\log_2(40) - \log_2(3) \approx 5.3 - 1.6 = 3.7$ bits of information per observation. This loss is **intentional and justified**: the three color classes provide glanceable action (red → hire, amber → watch, green → maintain) at the cost of precision. Formal information loss computation using Shannon entropy requires the full distribution of PPH values, which varies by belt family and phase.

10.3 Volume Reconciliation as a Consistency Check

The Volume Reconciliation panel compares:

$$V_{\text{iGate}} = \sum_{b \in B} (\text{LooseOutbound}_b + \text{BagsLinked}_b)$$

against:

$$V_{\text{SOR}} = \sum_{a \in A} (\text{Planned Vol}_a \text{ or Actual Vol}_a)$$

The **status** is:

$$\text{Status} = \begin{cases} \text{IN AGREEMENT} & |V_{\text{iGate}} - V_{\text{SOR}}| < \epsilon \\ \text{MINOR VARIANCE} & \epsilon \leq |V_{\text{iGate}} - V_{\text{SOR}}| < 2\epsilon \\ \text{INVESTIGATE} & |V_{\text{iGate}} - V_{\text{SOR}}| \geq 2\epsilon \end{cases}$$

with $\epsilon = 100$ packages (a tunable tolerance). This is a **hypothesis test** at each snapshot: H_0 is "systems are synchronized", H_A is "data fidelity has drifted".

---

11. [TRACKER] Sort Phases as a Temporal Markov Chain

11.1 Phase Definition

A sort operates in phases with distinct structural properties:

1. **Ramp (18:00–19:00)** — Inbound volume arrival, stagger staffing ramp-up, early PPH unreliable
2. **Steady-State (19:00–21:00)** — High volume, stable staffing, representative PPH
3. **Wind-Down (21:00–22:00)** — Volume declining, staff consolidation, borrow/loan cross-area activity
4. **Post-Sort (22:00+)** — Wrapping, reconciliation, no operational decisions

Define a **phase state** $\phi(t) \in \{\text{Ramp}, \text{Steady}, \text{Wind}, \text{Post}\}$ as a function of elapsed time t.

11.2 Phase-Dependent Statistics

The snapshot S_t is characterized not just by time but by phase:

$$\Pr[\text{PPH}_{kb}^{(t)} > \text{Target} | \phi(t) = \text{Ramp}] < \Pr[\text{PPH}_{kb}^{(t)} > \text{Target} | \phi(t) = \text{Steady}]$$

The Ramp phase has **non-stationary distribution** due to stagger staffing (intentional understaffing). A Ramp-phase PPH alarm is a **false positive** — structurally expected, not a performance failure.

11.3 Staffing as a Counting Process

Within each phase, the number of active employees $N(t)$ follows a **counting process** (e.g., Poisson-like with arrivals at punch-in and departures at punch-out). The **intensity** $\lambda(t) = dN/dt$ indicates:

- $\lambda(t) > 0$ during Ramp (staffing arrivals)
- $\lambda(t) \approx 0$ during Steady (stable headcount)
- $\lambda(t) < 0$ during Wind (departures / borrow consolidation)

The **borrow/loan event** (z_i, z_j, t_b) — moving one employee from zone i to zone j at time t_b — is a **discontinuity** in $N(t)$ per zone. Tracking these as a point process reveals **demand imbalance patterns** across sorts.

[**OPEN**] Build a historical borrow event database: timestamp, source zone, dest zone, impact on both zones' PPH. Correlate against pred-sort DOP and volume curve shape to model which belt configurations trigger borrows.

---

12. [TRACKER / DASHBOARD] PPH Quantization and Color Bands

12.1 The Quantization Hierarchy

The Tracker embeds PPH color bands at multiple levels:

1. **Employee-level** (Per-Employee Productivity table) — per belt, per employee
2. **Belt-level** (iGate Scan Metrics, Hub tab) — aggregating across all employees per belt
3. **Zone-level** (Area Outbounds tab) — aggregating across all employees per coordinator zone

Each level applies the quantization $q_F : \text{PPH} \to \{\text{red}, \text{amber}, \text{green}\}$ with belt-family-specific thresholds.

12.2 Aggregation is Non-Homomorphic

If PPH_{kb} is the per-employee PPH on belt b and $\text{PPH}_b = \frac{\sum_k n_{kb}}{\sum_k h_{kb}}$ is the belt-level aggregated PPH, then:

$$q_F(\text{PPH}_b) \neq \text{majority}(q_F(\text{PPH}_{kb})) \quad \text{(in general)}$$

Example: A belt with two employees — Employee A with 10 PPH (red) and Employee B with 35 PPH (green) — has aggregate:

$$\text{PPH}_b = \frac{10 + 35}{2} = 22.5 \quad \Rightarrow \quad q_F(22.5) = \text{amber}$$

But the majority color is red or green (1–1 tie). The aggregated belt color (amber) is determined by the **arithmetic mean**, not by voting. This is a **non-homomorphic aggregation** — information is lost and restructured during aggregation.

Implication: The Tracker's belt-level colors are *derived summaries*, not direct measurements. They reflect the aggregate operating point, not a vote among employees. The per-employee breakdown is the foundational data; belt colors are coarse visual summaries for tactical decision-making. Supervisors should cross-reference colors with the underlying per-employee detail table to understand why a belt is red, amber, or green.

12.3 Color Bands as a Quotient Space

The partition of $\mathbb{R}^+$ into three bands $\{\text{red}, \text{amber}, \text{green}\}$ defines an equivalence relation:

$$a \sim_F b \iff q_F(a) = q_F(b)$$

The **quotient space** $\mathbb{R}^+ / \sim_F$ has three equivalence classes. Operations on PPH values are, in some sense, operations on equivalence classes:

- "Is this belt performing?" $\Leftrightarrow$ "Is its PPH in the green class?"
- "Which belt needs help?" $\Leftrightarrow$ "Which belt's PPH is in the red class?"

[**OPEN**] Experiment with **adaptive thresholds** based on staffing size and phase. Small-crew ramp phases should have lower green thresholds than steady-state large-crew phases, to account for the structural per-phase variation.

---

13. [MASTER TREND ANALYSIS] The Data Extraction Pipeline as a Functor

13.1 The Extractor Signature

The **MasterTrendAnalysis** project (v2.0, 620-sort historical corpus) processes historical tracker workbooks via an extraction pipeline:

$$\text{Extract} : \text{Path}_{\text{.xlsx}} \to \text{Json}_{\text{sort}}$$

The function reads an Excel workbook and outputs a JSON record with schema version `'v1.7'`:

$$\text{Json}_{\text{sort}} = \{ \quad \text{date},\ \text{dow},\ \text{era},\ \text{adjusted\_pph},\ \text{volume}, \quad \text{hours\_worked},\ \text{ids\_share},\ \text{door\_turns},\ \text{belt\_stops}, \quad \text{timeline} : \text{Jso}$$

Each $\text{Json}_{\text{timeline}}$ record is a snapshot S_t (cf. Section 10) serialized to JSON:

$$\text{Json}_{\text{timeline}} = \{\text{elapsed\_min},\ \text{pph},\ \text{volume},\ \text{hours},\ \text{belt\_detail} : \text{Json}_{\text{belt}}\}$$

13.2 Label-Based Scanning as Pattern Recognition

Rather than relying on fixed row numbers (which shifts across tracker versions), the extractor uses **label-based scanning**: it searches for known cell values (e.g., "Adjusted PPH", "Volume", "Hours Worked") and derives the row number dynamically. This is a **robustness pattern** — the extractor is a functor that works across multiple domain types (different tracker versions).

Formally, define:

$$\text{Find}(\text{label}, \text{sheet}) \to (\text{row}, \text{col})$$

as a search function returning the first cell address containing label. The extraction of a value becomes:

$$\text{GetValue}(\text{label}, \text{sheet}) = \text{sheet}[\text{Find}(\text{label}, \text{sheet})]$$

This decouples the extraction from the layout — a **structural abstraction** that survives tracker reformats.

13.3 Schema Versioning and Migration

The JSON corpus has schema version `'v1.7'`. Historical versions include `'v1.6'` (no belt stops), `'v1.5'` (no IDS), etc. The extractor's output schema is controlled by the codebase version; older workbooks are **lifted** to the new schema:

$$\text{Json}_{\text{v1.6}} \xrightarrow{\text{Lift}} \text{Json}_{\text{v1.7}}$$

with missing fields populated as `null` or computed (e.g., IDS share backward-estimated from volume trends if not originally captured).

[**OPEN**] Implement a **schema migration roadmap** document showing which fields were added in which version, and whether backward-filling is possible or if data is permanently lost.

14. [MASTER TREND ANALYSIS] Historical Trend Inference and Prediction Algorithms

14.1 The Peer Group Selection as a Filtered Subspace

When predicting a sort's outcome, the MasterTrendAnalysis uses **peer group selection**:

$$\text{PeerGroup} = \text{Filter}(\text{Corpus}, \{\text{era},\ \text{dow},\ \text{ssStatus},\ \text{idsStatus}\})$$

This is a **topological refinement**: starting with the full 620-sort corpus, we restrict to a subspace defined by:

- **Era** $\in \{\text{pre\_sls}, \text{sls02}, \text{dual\_sls}\}$ — system configuration
- **DOW** $\in \{\text{Mon}, \ldots, \text{Fri}\}$ — day-of-week seasonality
- **SS Status** $\in \{\text{normal}, \text{partial}, \text{down}\}$ — sleeve sort machine state, classified via IQR Tukey fence on sls02/dual_sls peers
- **IDS Status** $\in \{\text{low}, \text{normal}, \text{high}\}$ — inbound sort status, classified via IQR on ids_share_pct

The filtered corpus PeerGroup typically contains 15–40 sorts — a **representative cohort** of similar operational conditions.

14.2 Curve Fitting Algorithms

Given a peer group, the MasterTrendAnalysis offers four prediction algorithms:

Normalized Curve (Alignment) Align each peer sort's PPH curve to a common timeline by phase: 1. Rescale each peer's ramp phase to [18:00, 19:00] 2. Overlay all ramp curves, compute percentile bands (P25, P50, P75) 3. Repeat for steady-state and wind-down phases 4. Composite prediction = phase-aware blend of peer percentile bands

This is a **piecewise affine registration** — accounting for the fact that ramp/steady/wind phases have different typical durations.

Weighted Median For each 15-min elapsed time bucket t_i, compute the weighted median PPH of the peer group, weighted by operational similarity (e.g., closeness in staffing level, volume trend). This is a **robust estimator** — median is less sensitive to outliers than mean.

KDE (Kernel Density Estimation) For each peer sort p and time t_i, extract $\text{PPH}_p(t_i)$. The **KDE** models the distribution of PPH values across the peer group as:

$$\hat{f}(x) = \frac{1}{nh} \sum_{p=1}^{n} K\left(\frac{x - \text{PPH}_p(t_i)}{h}\right)$$

where K is a Gaussian kernel and h is the bandwidth. The prediction at confidence level α is the $(1-\alpha)$ quantile of the estimated distribution.

Bandwidth Selection (v1.2 research update): The current implementation uses **Silverman's rule of thumb**:

$$h_{\text{Silverman}} = 1.06 \cdot \sigma \cdot n^{-0.2}$$

This is simple and computationally efficient ($O(n)$), but for non-normal or multimodal PPH distributions, the **Sheather-Jones plug-in method** provides better accuracy. Sheather-Jones directly estimates the optimal bandwidth by minimizing mean integrated squared error (MISE), making it superior when peer groups exhibit multiple modes (e.g., by belt family or operational phase). For production deployments: benchmark both on your peer corpus—Silverman if speed dominates and data is roughly unimodal, Sheather-Jones if accuracy is critical and budget allows the $O(n^2)$ computation.

DTW (Dynamic Time Warping) DTW measures distance between the current partial sort's PPH curve and each peer's full curve, allowing for **temporal distortion** (peers that have similar shape but shifted in time):

$$\text{DTW}(\text{current}, \text{peer}) = \min_{\text{path}} \sqrt{\sum_{(i,j)\in\text{path}} (\text{PPH}_{\text{current}}(i) - \text{PPH}_{\text{peer}}(j))^2}$$

This is a **dynamic programming** problem with $O(mn)$ complexity. The algorithm selects the top-20 closest peers by DTW distance and uses their future trajectories to project the current sort.

Optimization via Sakoe-Chiba Band (v1.2 research update): To accelerate DTW computation, a **Sakoe-Chiba band constraint** limits the warping path to a band of radius r around the main diagonal of the cost matrix. This reduces complexity to $O(m \cdot r)$, significant when $r \ll n$. The constraint is valid here because sort PPH curves are expected to have small variance in temporal alignment—a sort that's slow early is likely slow throughout. Set $r \approx 0.1 \cdot n$ (e.g., $r = 24$ for a 240-minute sort) to allow $\pm 10\%$ temporal flexibility while avoiding pathological warps. Benchmark on your peer corpus to tune r.

14.3 Baseline: Multiple Linear Regression (MLR)

As a "null model" for comparison, the MasterTrendAnalysis fits an OLS regression on the dual_sls corpus:

$$\text{Adjusted PPH} = \beta_0 + \beta_1 \cdot \text{DOW}_{\text{code}} + \beta_2 \cdot \text{SS Share} + \beta_3 \cdot \text{IDS Share} + \beta_4 \cdot \text{Vol/Head} + \beta_5 \cdot \text{Is Peak} + \epsilon$$

where DOW is coded as an indicator (or multiple indicators for multi-level categories).

Coefficients are estimated via **Gauss-Jordan inversion** of the normal equations.

Feature Scaling (v1.2 research): Continuous features (SS Share, IDS Share, Vol/Head) should be **centered** (subtract mean) to reduce structural multicollinearity, especially if interaction terms are added. Do *not* standardize across continuous and categorical mixed features—categorical variables cannot be standardized (no mean/stdev). If you add interaction terms like DOW × SS Share, centering the continuous feature avoids multicollinearity inflation.

The MLR model provides **percentile scenarios** (P25/median/P75) based on the peer group's feature distribution — a baseline expectation for how much of the final PPH variance can be explained by pre-sort observable features. **Future work**: add interaction terms (e.g., DOW × Vol/Head, Peak × SS Share) and cross-validate to check whether they improve RMSE.

14.4 Kalman Filter for Sequential Updating

As new snapshots arrive during the sort, the MasterTrendAnalysis maintains a **Kalman filter** state:

$$\hat{x}_t = \text{Adjusted PPH estimate at elapsed time } t$$

The filter has: - **State model:** $x_t = x_{t-1} + w_t$ where $w_t \sim \mathcal{N}(0, Q)$ (random walk model; PPH undergoes slow random fluctuations around an unknown mean level, not systematic drift) - **Observation model:** $z_t = x_t + v_t$ where $v_t \sim \mathcal{N}(0, R(t))$ (measurement noise from incomplete/partial data at each snapshot) - **Process noise covariance:** $Q = 5\% \times$ corpus variance (a tuned hyperparameter, small to discourage large PPH swings) - **Measurement noise covariance:** $R(t) = R_{\text{base}} \times (1 - t/240)^2$ (variance decreases as sort progresses and more aggregated data arrives; near sort-end, measurement becomes more precise)

At each new snapshot, the filter updates:

$$K_t = \frac{P_t^-}{P_t^- + R(t)} \quad \text{(Kalman gain)}$$

$$\hat{x}_t = \hat{x}_{t-1} + K_t(z_t - \hat{x}_{t-1})$$

This is **optimal sequential estimation** under Gaussian assumptions — early estimates are uncertain, late estimates are precise.

Cold-Start Initialization (v1.2 research): A critical practical problem is initializing the filter at the first snapshot with no prior history. Research shows three effective strategies:

1. **Method 1 (Current):** Use peer group mean as initial state $x_0 = \bar{\text{PPH}}_{\text{peers}}$, set P_0 (initial uncertainty) large to ensure rapid convergence. **Pro:** simple. **Con:** transient lasting 30–60 min before convergence.
2. **Method 2 (Bayesian):** Treat x_0 as a random variable and use Bayesian inference to estimate it jointly with future states, using the first few measurements to refine the posterior. **Pro:** reduces transient. **Con:** higher computational cost.
3. **Method 3 (Gaussian Process Optimization):** Off-line benchmark the Kalman filter on historical sorts to find optimal $(P_0, Q, R_{\text{base}})$ parameters that minimize RMSE on held-out test sorts. Research shows 80%+ reduction in prediction error vs. naive initialization. **Pro:** best empirical performance. **Con:** requires corpus of test sorts.

Recommendation for v1.2+: Implement Method 1 for production (simplicity), then benchmark Methods 2 & 3 offline to see if the overhead is justified by RMSE improvement on your 620-sort corpus.

14.5 Prediction Error Analysis and Algorithm Selection

The MasterTrendAnalysis post-hoc evaluates prediction error:

$$\text{Error}_{\text{sort,algo}} = |\text{Predicted PPH} - \text{Actual PPH}|$$

Aggregate across the corpus:

$$\text{MAE}_{\text{algo}} = \frac{1}{|\text{Corpus}|} \sum_{\text{sort}} \text{Error}_{\text{sort,algo}}$$

$$\text{RMSE}_{\text{algo}} = \sqrt{\frac{1}{|\text{Corpus}|} \sum_{\text{sort}} \text{Error}^2_{\text{sort,algo}}}$$

Algorithms are ranked by MAE/RMSE; the **algorithm selector dropdown** defaults to the best-performing variant for the selected era/DOW/conditions. This is **adaptive model selection** — the tool learns which method works best and surfaces it.

---

15. [MASTER TREND ANALYSIS] Door Turns as a Leading Indicator

15.1 Inbound Flow Process

The Hub has 38 active inbound bays receiving packages at rates of 800–1,200 packages/hour per door group. A **door turn** is a single carriage of packages arriving at a bay. The total door turns in a sort is a **proxy for total inbound volume**, measured earlier than the final hub summary (which lags by ~20 min after the last scan).

The **door ratio** at elapsed time t is:

$$\text{DoorRatio}(t) = \frac{\text{Doors to Date}(t)}{\text{Doors Forecast}(t)}$$

For a peer group, we compute the historical median door ratio at each time bucket, giving us a **leading indicator** of whether the sort is ahead or behind volume pace:

$$\text{Ahead if DoorRatio}(t) > 1.08, \quad \text{Behind if DoorRatio}(t) < 0.92$$

This is a **threshold-based control signal** — wider than a single number because door counting has noise.

[**OPEN**] Correlate door ratio at 19:00 with final PPH across 620 sorts. If correlation is strong (>0.6), use door ratio as a **live input** to the Kalman filter, improving mid-sort predictions.

---

16. [MASTER TREND ANALYSIS] The Sort Summary as a Narrative + Heatmap

16.1 Phase-Stratified KPI Table

The Sort Summary tab decomposes the 4-hour sort into 4 operational phases:

Phase	Time Window	Typical Staffing	Volume Characteristic
Ramp-Up	18:00–19:00	Understaffed (stagger)	Early inbound spike
Peak Ops	19:00–20:15	Full	High sustained volume
Power Hour	20:15–21:15	Full or slightly reduced	Lingering inbound + final volume surge

Phase	Time Window	Typical Staffing	Volume Characteristic
Wrap/PD-04	21:15–22:00	Consolidated	Tail volume, wrapping

For each phase, the table shows:

$$\text{PPH}_{\text{phase}} = \frac{\sum_{\text{sort}\in\text{phase}} \text{volume}}{\sum_{\text{sort}\in\text{phase}} \text{hours}}$$

along with peer-group percentile bands (P25/P50/P75). A **decision guide** annotates each phase with actions:

- Staff: "Add people if volume is running hot"
- Cut: "Send people home if lagging"
- Watch: "Monitor closely, may need borrow"
- PD-04: "Wrap time warning"

This is **operational decision support** — the table surfaces which phase is critical and what action is available.

16.2 The Sort Heatmap as a Delta-Baseline Matrix

The **Sort Heatmap** (Area View tab, v1.10+) shows a 3-mode representation of time-series PPH:

Mode 1: Volume Absolute Cell (t, z) = volume in zone z at time t, in packages. Color ramp: cream → brick → red (intensity).

Mode 2: Volume Delta Cell (t, z) = (volume in zone z at time t) — (median peer volume at time t). Color: blue (below median) warm (above median). This isolates **zone behavior relative to peers** — a zone that's 10% below is blue; 10% above is red.

Mode 3: Belt PPH Cell (t, b) = PPH on belt b at time t. Color: cream (cold) → gold (warm) → red (hot). Tuned to the peer group's distribution.

The heatmap is a **multidimensional data view** — three different coordinate projections of the same underlying data cube $(t, z, b, \text{PPH}, \text{volume})$. Switching modes reveals different patterns: volume imbalances vs performance imbalances.

---

17. [TRACKER + MASTER TREND] Three Timescales and Their Relationship

17.1 Hierarchy of Observation Granularities

The Hub Operations ecosystem operates at three distinct timescales:

Timescale	Granularity	Project	Example Data
Per-Question	1–5 seconds	Label Training Cert (LTC)	Sorter answer, routing function, correct/incorrect

Timescale	Granularity	Project	Example Data
Per-Snapshot	15–30 minutes	Tracker (v2.9)	iGate/SOR aggregates, belt-level PPH, staffing state
Per-Sort	4 hours (240 min)	MasterTrendAnalysis	Final adjusted PPH, volume, era, staffing efficiency

17.2 Upward Aggregation

Observations aggregate upward with **information loss** at each level:

Level 1 → Level 2 (Training to Intra-Sort):

$$\text{Snapshot}_t \supset \{\text{sorter}_k : t_k^{\text{last}} \leq t\}$$

Individual sorter training events are not directly visible in the tracker (which focuses on operational package sorts, not training); but if a sorter is weak on a belt (detected in training), their belt-specific PPH in the tracker should reflect it. **Gap:** LTC currently has no bidirectional link to the Tracker — training insights are advisory, not fed back into operational dashboards.

Level 2 → Level 3 (Intra-Sort to Historical):

$$\text{SortRecord} = \int_0^{240} S_t \, dt$$

where S_t is the snapshot at time t. The integral (in the Lebesgue sense) aggregates all snapshots into a single sort record. Information loss includes:

- Temporal phase structure (we collapse 4 hours to 1 number: final adjusted PPH)
- Employee-level detail (we aggregate to belt and area)
- Intra-sort variance (we keep only the final state, losing the trajectory uncertainty)

17.3 Downward Reconstruction (Prediction)

Conversely, predictions flow downward:

Level 3 → Level 2 (Historical to Intra-Sort):

The MasterTrendAnalysis predicts a sort's **trajectory** $\hat{S}(t)$ based on peers. For each peer sort, we have the full timeline $S(t)$ for $t \in [0, 240]$. The peer group median or KDE estimate reconstructs a predicted trajectory:

$$\hat{S}(t) = \text{KDE}_t(\{\text{peer}(t) : p \in \text{PeerGroup}\})$$

This predicted trajectory is sent to the Tracker's Forecast tab (v4.2+), where it displays as a "DOP predictor" or "live pace check" overlay against actual snapshots.

Level 2 → Level 1 (Intra-Sort to Per-Question) — FUTURE:

A natural extension would be to use intra-sort zone/belt analytics to **generate targeted training** for the next session. E.g., "Zone 3 had 8% below-target PPH on Belt 7; next LTC session should emphasize Belt 7 routing rules for that zone's sorters." This is a **feedback loop** not yet implemented.

---

18. [GLOBAL] Filtered Spaces and Stratification

18.1 Corpus Filtering as Topological Refinement

Both the Tracker and MasterTrendAnalysis filter data based on user selection:

- **Tracker:** Time range (18:00–22:00, or a sub-window), zone/belt selection (which areas to display)
- **MasterTrendAnalysis:** Era, DOW, peak season, SS/IDS status, specific date range

A **filter** φ is a function:

$$\varphi : \text{Domain} \to \{0, 1\}$$

selecting a subset $\{x : \varphi(x) = 1\}$. The filtered domain is a **topological subspace** (inherited topology from the original domain).

18.2 Stratification by Era

The 620-sort corpus is stratified into three eras based on operational configuration:

- **pre_sls** (~236 sorts, pre-2024): No sleeve sort machine
- **sls02** (~178 sorts, 2024–early 2025): Sleeve sort v0.2
- **dual_sls** (~206 sorts, 2025–present): Dual-sleeve configuration

Each era has **distinct statistical distributions** (e.g., final PPH, door turn rates, staffing curves). A cross-era analysis is misleading: PPH in pre_sls is not comparable to PPH in dual_sls due to operational differences.

The **era filter** is a **stratification variable** — it partitions the corpus into blocks with homogeneous operational parameters. Analysis must respect strata, or risk Simpson's Paradox (a trend within each stratum reverses in the combined data).

18.3 Seasonal Stratification

Within each era, the corpus is further stratified by **peak season** (holiday shopping, predictable annual spikes). Peak sorts have:

- Higher inbound volume
- Different staffing profiles (more borrowing, stagger staffing earlier)
- Different phase structure (Power Hour may arrive earlier or extend longer)

A sort's **peak status** is encoded as `is_peak = 1` if it falls in Nov/Dec or other designated peak windows, 0 otherwise.

[OPEN] Quantify the statistical difference between peak and non-peak within each era. Does the MasterTrendAnalysis's default algorithms work well for peak prediction, or does it require a separate model?

---

19. [LABEL TRAINING CERT] Knowledge Transfer as Information Geometry

19.1 The Learner's State Space

A sorter's state of knowledge can be modeled as a **point in a high-dimensional space**. The dimensions are:

- Per-belt routing accuracy: $(\theta_b)_{b \in B}$, where $\theta_b = \Pr[\text{correct}|\text{belt} = b]$
- Per-state regional knowledge: $(\theta_s)_{s \in \text{States}}$
- Confusion confusion-graph distance: how much do they confuse similar belts?

The sorter's **trajectory** through this space over repeated quizzes is a learning curve:

$$\theta_k(t) : t \in \{1, 2, \dots, n_k\} \to \mathbb{R}^{|B|+|\text{States}|}$$

A sorter who improves is moving toward the **ideal point** $\theta^* = (1, 1, \dots, 1)$ (perfect on all dimensions).

19.2 Information Geometry Perspective

In information geometry, the **Fisher information matrix** $\mathbf{I}(\theta)$ measures the curvature of the likelihood landscape:

$$\mathbf{I}_{ij}(\theta) = \mathbb{E}\left[\frac{\partial \log \Pr}{\partial \theta_i} \frac{\partial \log \Pr}{\partial \theta_j}\right]$$

High curvature (high $\mathbf{I}$) means the likelihood is "peaked" — the sorter's true skill can be estimated precisely from few trials. Low curvature means wide uncertainty — many trials needed.

For a sorter with n trials on each of $|B|$ belts (uniform sampling):

$$\mathbf{I} \approx n \cdot \text{diag}\left(\frac{1}{\theta_1(1-\theta_1)}, \dots, \frac{1}{\theta_{|B|}(1-\theta_{|B|})}\right)$$

The Fisher information for each belt is inversely proportional to the variance of the Bernoulli distribution. Belts where $\theta_b \approx 0.5$ have maximal uncertainty (largest Fisher information, hardest to learn precisely — many trials needed). Belts where $\theta_b \approx 0$ or 1 have low Fisher information (easy to distinguish from uninformed guessing, but hard to improve beyond that). This suggests an **adaptive sampling strategy**: ask more questions on belts near $\theta_b = 0.5$ (maximum uncertainty) rather than on belts where the sorter is already proficient or hopeless.

---

20. The Sort Chart as a Finite Automaton

10.1 The Original Source Structure

The truth table was derived from a Sort Charts Master spreadsheet mapping ZIP → SLIC → Belt. This pipeline defines a **two-step composed function**:

$$Z \xrightarrow{g} \text{SLIC} \xrightarrow{h} B, \quad f = h \circ g$$

where SLIC (Sort Location Identification Code) is an intermediate routing symbol. The pipeline resolved ambiguous SLICs (multiple SLICs with the same name) and documented unmatchable rows.

10.2 Automaton Interpretation

Viewing the routing lookup as a **decision process**, the 3-digit ZIP prefix determines the SLIC, and the SLIC determines the belt. This is a **deterministic finite transducer** with: - Input alphabet: $\{0, \ldots, 9\}^5$ (ZIP digits) - State space: SLIC names (intermediate) - Output: Belt index in B - Transition function: first 3 digits $\to$ SLIC; SLIC $\to$ Belt

The "truth edits" system (`truth_edits.jsonl`) applies **state-transition overrides** — rewriting individual arcs in the transducer without rebuilding the whole automaton.

---

21. The Probabilistic Skill Model (Bayesian Extension)

11.1 Beta-Bernoulli Model

A natural Bayesian model treats each sorter's true accuracy θ_k as a random variable with a **Beta prior**:

$$\theta_k \sim \text{Beta}(\alpha, \beta)$$

After observing c correct out of n attempts, the posterior is:

$$\theta_k | c, n \sim \text{Beta}(\alpha + c,\ \beta + n - c)$$

With a uniform prior ($\alpha = \beta = 1$), the posterior mean is:

$$\mathbb{E}[\theta_k | c, n] = \frac{c+1}{n+2}$$

This is the **Laplace-smoothed accuracy** — it shrinks extreme estimates towards 0.5, which is more appropriate for small n.

11.2 Hierarchical Model for Belt-Specific Skill

A sorter's skill is not uniform across belts. A **hierarchical Bayesian model** would parameterise:

- Global sorter skill: $\theta_k \sim \text{Beta}(\alpha_0, \beta_0)$
- Per-belt skill: $\theta_{k,b} \sim \text{Beta}(\alpha(\theta_k), \beta(\theta_k))$

The belt-specific posterior, updated with the per-belt breakdown from `sorterBeltBreakdown()`, enables more precise coaching recommendations: a sorter who is globally proficient but has 60% accuracy on Top Brown needs a specific intervention, not general retraining.

[OPEN] The hierarchical model is also the correct framework for the Knowledge Map: each sorter has a latent skill vector $(\theta_{k,s})_{s \in \text{States}}$ indexed by state. The observable data (accuracy per state) are noisy estimates of this vector. A proper ranking should account for uncertainty — the Wilson lower bound (Section 6.2) is a frequentist approximation of this idea.

---

22. Comprehensive Summary Table of Mathematical Structures

[**LABEL TRAINING CERT**]

Component	Mathematical Object	Key Property
ZIP-to-belt mapping	Partial function $f : Z \to B$	Locally constant at 3-digit prefix
CCHIL routing	Multifunction $\tilde{f} : Z \to \mathcal{P}(B)$	Acceptance relation, set-valued
ZIP code space	Ultrametric space (Z, d)	$d(x, z) \leq \max(d(x, y), d(y, z))$
Belt set with confusion	Directed graph $G = (B, E)$	Encodes hard-negative structure
Question selection	Categorical mixture distribution	4-pool weighted sampling
Sorter channel	Right stochastic matrix $\mathbf{M}$	Perfect sorter $\Rightarrow \mathbf{M} = \mathbf{I}$
Accuracy	Binomial proportion $\hat{\theta}$	Wilson CI for uncertainty
Trend	Two-proportions test	Formal non-stationarity check
Overlay system	Pointed function update $f \lhd o$	Left-regular band
Analytics reducers	Monoid homomorphisms $\mathcal{E}^* \to$ Stats	Enables MapReduce
Knowledge map filter	Boolean algebra $\mathcal{P}(\mathcal{E})$	OR-composition of state indicators
Sorter skill	Beta-Bernoulli posterior	Bayesian shrinkage for small n
Pipeline composition	Finite transducer $g \circ h$	Two-step ZIP $\to$ SLIC $\to$ Belt
Learner state space	Euclidean manifold $\mathbb{R}^{\lvert B\rvert+\lvert\text{States}\rvert}$	Learning curve as trajectory
Fisher information	Metric tensor $\mathbf{I}(\theta)$	Curvature of skill space

[**TRACKER v2.9 + DASHBOARD v2.9.2**]

Component	Mathematical Object	Key Property
Snapshot	Temporal tuple $(t, \mathcal{H}_t, \mathcal{E}_t, \mathcal{A}_t)$	Discrete observation, 4-tuple structure
Snapshot sequence	Time-indexed series $\mathcal{S} = (S_{t_1}, \ldots, S_{t_m})$	0 t 240 min, 4-hour sort window
Per-employee PPH	Ratio of linear functionals $\frac{n_{kb}}{h_k}$	Non-homomorphic under aggregation
PPH quantization	Quotient map $q_F : \mathbb{R}^+ \to \{R, A, G\}$	Belt-family-stratified color bands
Volume reconciliation	Consistency hypothesis test	Status: IN AGREEMENT / MINOR / INVESTIGATE
Sort phases	Piecewise constant phase function $\phi(t)$	Ramp / Steady / Wind / Post
Staffing dynamics	Counting process $N(t)$ with jumps	Intensity (t) $>$ 0 (Ramp), 0 (Steady), $<$ 0 (Wind)
Borrow/loan event	Point process on (z_i, z_j, t_b)	Zone transfer recorded as discontinuity

Component	Mathematical Object	Key Property
Belt-level aggregation	Weighted sum with non-homomorphic color mapping	Color(aggregate) majority(colors)
Data reconciliation gap	Absolute difference $\|V_{\text{iGate}} - V_{\text{SOR}}\|$	Tunable tolerance for drift detection

[MASTER TREND ANALYSIS v2.0]

Component	Mathematical Object	Key Property
Extraction functor	$\text{Extract} : \text{Path}_{.\text{xlsx}} \to \text{Json}_{\text{sort}}$	Label-based scanning, robust to layout changes
Schema versioning	Migration map $\text{Json}_v \xrightarrow{\text{Lift}} \text{Json}_{v+1}$	Backward-compatible lifting, nullable fields
Corpus	620-sort dataset, stratified by era & DOW	pre_sls / sls02 / dual_sls eras
Peer group	Filtered subspace via $(era, dow, ssStatus, idsStatus)$	Topological refinement, 15–40 sorts typical
Normalized curve	Phase-aware PPH percentile bands $(P25, P50, P75)$	Piecewise affine registration by phase
Weighted median	Robust estimator, weighted by operational similarity	Less sensitive to outliers than mean
KDE (Kernel Density Est.)	$\hat{f}(x) = \frac{1}{nh} \sum_p K\left(\frac{x - PPH_p}{h}\right)$	Silverman bandwidth $h = 1.06\sigma n^{-0.2}$
DTW distance	Dynamic time warping, $O(620 \times 64)$ computation	Top-20 closest peers by DTW selected
MLR baseline	OLS regression, 5 features (DOW, SS share, IDS share, vol/head, peak)	Gauss-Jordan inversion, P25/med/P75 scenarios
Kalman filter	State $x_t \sim \mathcal{N}(\hat{x}_t, P_t)$, gain $K_t = \frac{P_t^-}{P_t^- + R(t)}$	Process noise Q = 5% corpus var, R(t) $\to$ 0 as t $\to$ 240
Door ratio	Leading indicator $\frac{\text{Doors}(t)}{\text{Doors Forecast}(t)}$	Ahead/Behind thresholds $\pm 8\%$
Phase-stratified table	4 phases $\times$ (PPH, vol, hours, staff)	Ramp / Peak Ops / Power Hour / Wrap
Sort heatmap	3 modes: volume absolute, volume delta, belt PPH	Coordinate projections of (t, z, b) cube
Era stratification	Partition corpus by operational config	Simpson's Paradox guard, 3 strata
Peak season flag	Binary classifier is_peak $\in \{0, 1\}$	Annual holiday spikes, affects staffing
Prediction error	MAE / RMSE per algorithm, ranked	Algorithm selection via empirical performance

23. Integration Points and Feedback Loops (Future Architecture)

23.1 LTC ↔ Tracker Bidirectional Link

Current state: LTC is standalone. Sorter training events don't feed back to the Tracker dashboard.

Future architecture: When a sorter completes LTC training, export per-belt accuracy to a tracker-side module:

$$\text{LTC Export} = \{(k, b, \theta_{kb}, \text{date}) : \text{sorter } k,\ \text{belt } b,\ \text{estimated accuracy } \theta_{kb}\}$$

The Tracker's roster view could annotate each employee with their known per-belt weakness:

$$\text{CoachingFlag}_k = \arg\min_b \theta_{kb} \quad \text{if } \theta_k < 0.85 \text{ overall}$$

This surfaces "who needs help and on what belt" at shift start, enabling **targeted coaching before the sort**, not post-hoc analysis.

Mathematical model: A sorter's true operational accuracy $\theta_k^{\text{ops}}(t)$ on belt b is a function of: - Training-measured skill θ_{kb}^{LTC} (from LTC database) - Operational stress (fatigue, time-of-shift, workload) - Recency decay (skill measured 3 days ago vs measured this morning)

$$\theta_k^{\text{ops}}(b, t) \approx \alpha\theta_{kb}^{\text{LTC}} + \beta \cdot f_{\text{stress}}(t) + \gamma \cdot e^{-\lambda(t - t_{\text{measure}})}$$

A **Bayesian hierarchical model** would combine prior (LTC accuracy) with observed operational data (from Tracker snapshots) to estimate true skill in real-time.

23.2 Tracker ↔ MasterTrendAnalysis Prediction Loop

Current state: Tracker shows iGate/SOR data live; MasterTrendAnalysis predicts offline, serves as a planning tool.

Future architecture: The MasterTrendAnalysis's Kalman filter state and peer-group predictions are sent to the Tracker in real-time via:

$$\text{PredictionUpdate}_t = \{\text{predictedPPH}_{\text{algo}},\ \text{P25},\ \text{P50},\ \text{P75},\ \text{dop\_tracker}_{\text{predicted}}\}$$

The Tracker's DOP Calc tab (Section 10 of tracker_v2.9.html) displays this as a live **pace overlay**:

- Red zone if actual < P25 (running slow)
- Green zone if actual within [P25, P75] (on pace)
- Blue zone if actual > P75 (running hot, ahead of pace)

This enables **in-sort tactical decisions** without manually checking the prediction tool.

23.3 Dashboard Post-Sort Reconciliation

Current state: Dashboard (v2.9.2) is a post-sort reporting tool; reconciliation is manual.

Future architecture: Implement a **three-stage post-sort workflow**:

1. **Fast Triage (15 min after sort-down):** Automated flagging of:
   - Employees with iGate scans not matching SOR punch area
   - Late punch-in gaps (First Scan Timestamp $>$ 18:15 $\rightarrow$ lost ramp-phase productivity)
   - Ghost employees (in SOR but zero iGate scans)
   - Jam-breaker misclassification (employees in "clk4"/"clk1" overhead codes needing reclassification)
2. **Data Fidelity Scoring:** Compute per-sort a composite score:

$$\text{FidelityScore} = 0.4 \times \text{SOR/iGate Alignment} + 0.3 \times \text{Ghost Employee Rate} + 0.3 \times \text{Late Punch In Share}$$

where each component is in $[0, 1]$. A sort with FidelityScore < 0.75 flags "data needs human review before analytics".

3. **Reconciliation Database:** Store all reconciliation decisions (who was reclassified as jam-breaker, which SOR entries were corrected) in an append-only log, enabling **long-term auditing** of data cleanup patterns.

---

24. Open Problems and Future Sessions

The following items are marked `[OPEN]` in the text above, plus integrated threads across projects:

LTC-Specific

1. **Preimage size analysis:** Characterise $|f^{-1}(b)|$ across belts. Which belts are underrepresented in training data purely by geography?
2. **Local constancy exceptions:** Find ZIPs where f is not locally constant at the 3-digit prefix level. These are training hard cases.
3. **Chromatic number of confusion graph:** What is the minimum number of distinct visual cues needed to make all belts unambiguous?
4. **Wilson CI ranking:** Add lower-bound sorting to the Sorters tab as an alternative ranking mode.
5. **Entropy column in Belt Analysis:** Add Shannon entropy H_i per belt to quantify *systematic* confusion versus low-frequency misses.
6. **Formal trend test:** Implement two-proportions z-test for the half-split trend comparison.
7. **Intersection filters:** Knowledge Map AND-combination: rank sorters by their weakest category, or by the product of category accuracies.
8. **CUSUM monitoring:** Implement a CUSUM chart for per-sorter accuracy over time, replacing or augmenting the sparkline.
9. **Bayesian hierarchical model:** Formalise the Beta-Bernoulli model per belt per sorter and evaluate whether it changes coaching recommendations in practice.
10. **Air sort internal routing:** When air sort slide sub-destinations (Blue Belt, Red Belt, Philly NDA, Louisville 2DA/3DA) are added, the air routing becomes a second-level routing function $f_{\text{air}} : (\text{rule} \times Z) \rightarrow B_{\text{air}}$ — another partial function on a product domain.

Tracker-Specific

11. **Adaptive PPH thresholds:** Per-phase, per-crew-size adaptive green/amber/red thresholds instead of fixed bands. Ramp-phase 18 PPH should not trigger red when small crew is expected.
12. **Borrow event database:** Historical timestamps, source/dest zones, impact on both zones' PPH. Correlate vs pre-sort DOP to model which configurations trigger borrows.
13. **Aggregate non-homomorphism analysis:** Quantify information loss in the belt-level PPH aggregation. When does a belt's color mislead vs reflect ground truth?

MasterTrendAnalysis-Specific

14. **Door ratio leading indicator correlation:** Correlate door ratio at 19:00 across 620 sorts with final PPH. If $r > 0.6$, use door ratio in Kalman filter.
15. **Peak season analysis:** Quantify statistical differences (PPH, staffing, duration) between peak and non-peak within each era. Does prediction model need stratification?
16. **Schema migration roadmap:** Document which fields were added per version, feasibility of backward-filling.
17. **Belt stops disruption model:** Implement IQR-based classification (low/normal/high/severe), incorporate as filter in prediction peer group.
18. **Algorithm per-era performance:** Which algorithm (Normalized Curve, Weighted Median, KDE, DTW) performs best in pre_sls vs sls02 vs dual_sls? Rank by RMSE per era.
19. **MLR feature engineering:** Add interaction terms (DOW × peak, SS share × vol/head). Cross-validate against test set.
20. **Kalman initialization:** Current assumes constant $x_0 = \bar{\text{PPH}}_{\text{peers}}$. Try initialization from MLR prediction instead.

Integration (Cross-Project)

21. **LTC Tracker bidirectional link:** Export sorter per-belt skill estimates from LTC, display on Tracker roster for coaching flags.
22. **Tracker MasterTrendAnalysis real-time loop:** Send Kalman filter predictions to Tracker's DOP Calc as live pace overlay (P25/P50/P75 bands).
23. **Post-sort reconciliation automation:** Fast triage of SOR/iGate mismatches, late punch-ins, ghost employees, jam-breaker misclassification. Fidelity score per sort.
24. **Three-timescale feedback closure:** Data from operational sorts (Level 3) enriches peer-group selection for future predictions; insights from training events (Level 1) inform coaching targets for sorters with known weaknesses.

---

25. How to Resume and Extend This Document

Before starting Session 3 or later:

1. Read the **Session Log** (top of this document) to know what was established.

2. Check the **Project State** section (now distributed across sections 10–22) to see if versions have advanced.
3. Read any section marked `[OPEN]` — these are incomplete and ready for investigation.
4. Add a row to the Session Log with today's date and the work you plan to do.

When extending:

- Add new sections following the pattern: [**PROJECT**] label, clear mathematical abstraction, connection to existing sections, and at least one `[OPEN]` problem.
- Update the Summary Table (§22) with any new structures.
- Update the Session Log immediately before submitting the document.
- Maintain the distinction between the three projects: LTC, Tracker, MasterTrendAnalysis. Use project context markers throughout.

For future sessions investigating integration:

- Start by re-reading section 23 (Integration Points) to understand what feedback loops are planned.
- Use the three-timescale hierarchy (§17) as a mental model for how data and insights flow.
- When implementing a feature, ask: "Does this connect to any of the three projects? Can I feed data back into another level?"

End of Session 2. This document is a living research artifact. Each session deepens specific sections, implements [OPEN] items, or discovers new mathematical structure. The goal is to make the three projects (LTC, Tracker, MasterTrendAnalysis, Dashboard) coherent under a unified mathematical framework.

Next session: pick up any [OPEN] item in priority order, implement it, and update the relevant sections with findings.